D Y PATIL UNIVERSITY
NAVI MUMBAI

Subject Name : Python Programming

Unit No:I

Unit Name :Introduction to Python Programming

Faculty Name : Swati Kadu

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Unit No.:I

Faculty Name :

Index

Lecture No & Topic Name	Slide No
	`

D Y PATIL UNIVERSITY
NAVI MUMBAI

Lecture No: 1

Introduction to Python Programming

1. Under the leadership of "ANDREW STUART TANENBAUM" a group of employees developed distributed operating system.

2. group of employees used ABC scripting language to develop distributed operating system.

3. "ABC" scripting language is very simple and easy to learn and work.

4. "GUIDO VAN ROSSUM" is a member in that group and he likes ABC scripting language very well as it is very simple and easy.

5. In Christmas holidays , Guido Van Rossum started developing a new language to be simpler and easy compare to ABC scripting language and all other existing language.

6. finally he developed a new language.

7. He likes "Nonty python's Flying Circus" English daily serial very well.

8. So finally Guido van Rossum developed python scripting language at the national research institute for mathematics & computer science in Netherland in 1989 and it available to public in 20th feb 1991.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Introduction to Python Programming

10. Python is now maintained by a core development team at the institute,although Guido Van Rossum still hold the vital role in directing its progress.

11. Python 1.0 was released in january 1994.

12. Python 2.0 was released in october 2000 any python 2.7.11 in the last edition of python 2.

13. Meanwhile, Python 3.0 was released in december 2008.

14. Python is general purpose high level programming language.

15. Python is recommended as first programming language for beginners.

16. Guido Van Rossum has developed python language by taking almost all programming features from different languages.

17.The most of the syntaxes in python is taken from C and ABC languages.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Python

- Python is general purpose programming language implemented by Guido Van Rossum in 1989 @ mathematics and science research centre called CWI.CWI is located @ nether land.

- Guido Van Rossum implemented python language by taking the different verities of language features like:
  - Procedure Oriented Programming Languages.
  - Object Oriented programming languages.
  - Scripting Languages.
  - Modular Programming Languages.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Python

-->Guido Van Rossum make it available python to public in 1991.

by using python we can implement different varities of application like :

1)Automation Application.

2) Data Analytics.

3)Web Application.

4)Scientific application.

5)Web Scrapping.

6)N/W with IOT.

7)Test Cases.

8)Gaming Application.

9) GUI Application.

10)Admin Application.

11)animation applications and so on.....

D Y PATIL UNIVERSITY
NAVI MUMBAI

Python

- Currently copy right of pythons are registered with an community & non profitable organisation called Python S/W foundation.
- Python S/W which is provided by python s/w foundation is known as "cpyhton".
- There are so many distributions are available for cpython

  1) Active Python.
  2) Anaconda Python.
  3) iPython.
  4)Pocket Python.
  5)PYPY Python.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Flavours of Python

1. **CPython:** It is the standard flaver of python. It can be used to work with c language applications.
2. **Jython or JPython:** It is for java applications. it can run on jvm.
3. **IronPython:** it is for C#.net platform.
4. **PYPY:** The main advantage of PYPY is performance will be improved because JIT compiler is available inside PVM.
5. **RubyPyt:** For Ruby Platforms.
6. **Anaconda Python:** It is specially designed for handling large volume of data processing.

-->The current version of CPython is 3.x(3.1,3.2,.....3.7)

-->Download the required python 3.7 version from "http://python.org" website

-->NTFS file system format in windows (No proper security,But it is user friendly)

-->APFS file system in MAC OS.

-->EXT3 file system in linux (user, group, author level security is provided in linux, but not user friendly.)

-->HDFS this file system is similar to linux but with limited differences. hon: For Ruby Platforms.

D Y PATIL UNIVERSITY
NAVI MUMBAI

How to convert one OS file to System to the anther file system?

1) By using 3rd party S/w like FileZilla we can convert one file into another.
2) By using Shares directories also we can convert one file system to another.

we can develop the python programs in two modes:

1)Interactive mode
2) Batch mode------>|--->Editors
|--->IDE's

D Y PATIL UNIVERSITY
NAVI MUMBAI

Interactive Mode:

- The concept submitting 1-by-1 python statement explicitly to the python interpritter is known as an interactive mode.
- We can submit 1-by-1 python statement explicitly to the python interpritter by using the python command line shell.
- We can open the python cmd line shell, by executing the python command on cmd prompt or terminal.

Drawbacks:

- Once if we exit from python cmd line prompt, the work which we done is going to loss.
- Interactive mode is suitable for learning python & to test predefined functions functionalities, but it is not suitable for application development.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Batch Mode

Concept of writting group of python statements in a file saving that file with extension.py file and submitting entire file to interpritter is known as batch mode.

-->We can develop the python files by using the "editors" or "IDE's".

-->Different editors are notepad, notepad++, editplus, nano, VI, GEdit etc...

-->Different IDE's are pycharm(90% usage), eclipse, netbeans, eric etc...

-->Open Notepad save file with .py in any drive and type code .

-->To run the program open cmd then type python filename.py or py filename.py

D Y PATIL UNIVERSITY
NAVI MUMBAI

Editors Dis-Advantages:

-->By using editors takes longer time(we have to remeber every thing while writting program).

-->By development the python files by using editors we can't use code generation tools (Smart help like ctrl+space)

-->We can't perform the automatic debugging operations on python files by using editors.

ID's Advantages: To overcome the problems we use IDE’s.

Different IDE's are:

-->Pycharm

-->Eric

-->Eclipse

-->Netbeans

-->Download and install pycharm

-->Open pycharm IDE by clicking on shortcut.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Features of Python

1).Simple and easy to learn:

-->Python is a simple programming language, when we read python program, we can fell like reading English statements.

-->The syntaxes are very simple and only 30+ keywords are available.

-->When compared with other languages , we can write programs with very less number of lines. Hence more readability and simplicity.

-->We can reduce development time and cost of the project.

2).Freeware & open source:

-->We can use python s/w without any license and it is a freeware.

-->It's source code is open, so that we can customize based on our requirement.

Ex: Jython is customized version of python to work with java applications.

D Y PATIL UNIVERSITY
NAVI MUMBAI

3).High level programming language:

-->Python is a high level programming language hence it is a programmer friendly

language. -->Being a programmer we are not required to concentrate low level activities like memory management and security etc....

4).Platform Independent:

-->Once we write the python program, it can run on any platform without rewritting once again.

-->Internally PVM is responsible to convert into machine understandable form.

5).Portability:

-->Python programs are portable. ie we can migrate from one platform to other platform very easily.

-->Python programs will provide same results on any platform.

D Y PATIL UNIVERSITY NAVI MUMBAI

6).Dynamically Typed:

-->In python we are not required to declare type for variables. -->Whenever we are assigning the value, based on value, type will be allocated automatically.

-->Hence python is considered as dynamically typed language.

-->But java, C are statically typed languages bcoz we have to provide type at the beginning only.

-->This dynamically typing nature will provide more flexibility to the programmer.

7).Both procedure oriented and object oriented:

-->Python language supports both procedure oriented(Like C, pascal etc) and object

oriented (Like C==, Java) features. -->Hence we can get benfits of both security & reusability.

8).Intrepreted:

-->We are not required to compile python programs explicitly. Internally python interpreter will take care that compilation.

DY PATIL UNIVERSITY NAVI MUMBAI

Features of Python

9).Extensible:

-->We can use other language programs in python.
-->The main advantage of this approach are:
1).We can use already existing legacy non-python code.
2).We can improve performance of the application.

10).Embeded:

-->We can use python programs in any other language program. -->i.e we can embedded python programs any where.

11).Extensive Library:

-->Python has a rich inbuilt library. -->Being a programmer we can use this library directly and we are not responsible to
implement the functionality.

Limitations of Python:

-->Performance wise not up to the mark bcoz it is interpreted language. -->Not suitable for mobile applications.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Understanding the Execution of Python Program

Python programs run through a set of internal steps that convert human-readable code into instructions the machine can understand.

Source code is not executed directly by the machine.

Python processes the code internally before execution.

This process allows the system to interpret and run programs correctly.

Example:

```
a = 10
b = 10
print(“Sum:", (a+b))
```

Output:

20

D Y PATIL UNIVERSITY
NAVI MUMBAI

Execution of a Python Program

During the execution of first.py, Python performs two main steps:

Step 1: Compilation to Bytecode
Python first checks your code for syntax errors. If the code is valid, it is converted into bytecode.

Bytecode is an intermediate, platform-independent format
It is not machine code
It represents instructions like calculations, comparisons, and memory operations
This bytecode is stored in a .pyc file inside a folder named __pycache__.

__pycache__/first.cpython-38.py

Python handles this process automatically, so you usually don't need to create .pyc files manually.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Step 2: Execution by Python Virtual Machine (PVM)

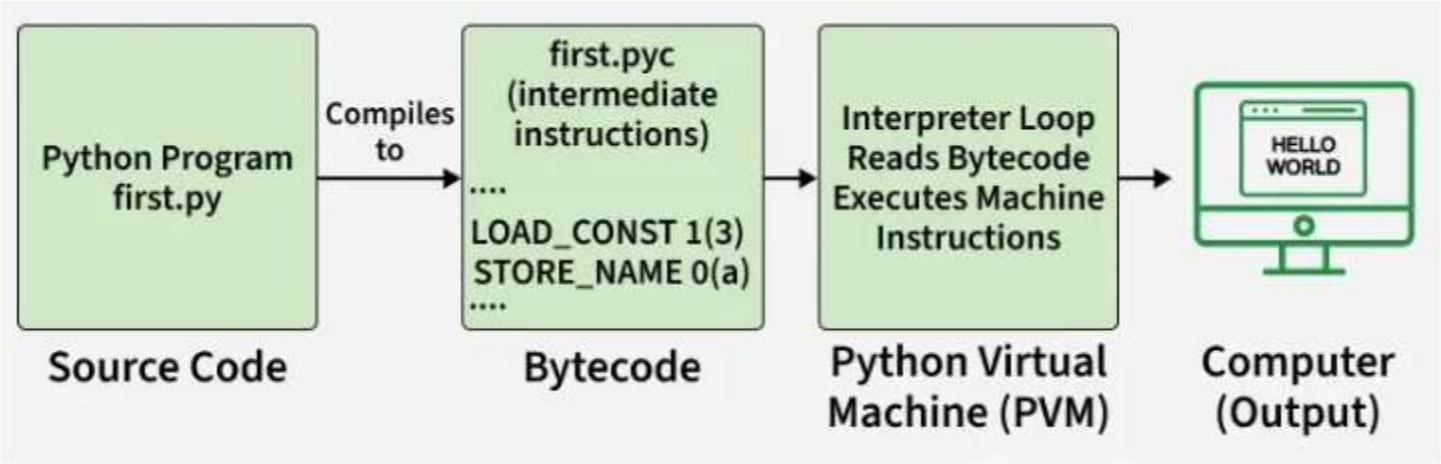
The Python Virtual Machine (PVM) reads the bytecode and executes it.

PVM understands the operating system and processor

It converts bytecode into machine-level instructions

These instructions are then executed by the CPU

This is why Python works on different platforms without changing the code.



Python: An Interpreted Language

The PVM executes bytecode instruction by instruction, not all at once. This line-by-line execution makes Python easy to debug but slightly slower than fully compiled languages.

About JIT (Just-In-Time) Compilation

To improve performance, some Python implementations use a Just-In-Time (JIT) compiler.

JIT converts frequently used bytecode into machine code

This improves execution speed

CPython (standard Python) does not use JIT

JIT is used in implementations like PyPy

D Y PATIL UNIVERSITY
NAVI MUMBAI

Viewing Bytecode Using dis

Python provides the dis module to inspect bytecode.

```
import dis
dis.dis("a = 10\nb = 10\nprint(a + b)")
```

The dis module (disassembler) shows:

Line numbers

Bytecode instructions

Operation names and arguments

This is mainly useful for learning and debugging, not for everyday programming.

```
0           0 RESUME                   0

1           2 LOAD_CONST               0 (10)
            4 STORE_NAME               0 (a)

2           6 LOAD_CONST               0 (10)
            8 STORE_NAME               1 (b)

3          10 PUSH_NULL
           12 LOAD_NAME                2 (print)
           14 LOAD_NAME                0 (a)
           16 LOAD_NAME                1 (b)
           18 BINARY_OP                0 (+)
           22 CALL                     1
           30 POP_TOP
           32 RETURN_CONST             1 (None)
```

Identifiers

-->A name in the python program is called as identifier.

-->It can be a class name or function or module name or variable name.

Ex: a=10

```
def f():
class test:
```

Rules to define identifiers in python:

1).The only allowed characters in python are

-->Alphabets(Either lower case or upper case)

-->Digits(0-9)

-->Underscore symbol(_)

-->By mistake if we are using any other symbol like $ then we will get syntax error.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Ex: cash=10(Valid)

ca$h=10(Invalid)

2).Identifiers should not start with digit.

Ex: 123total=500(Invalid)

total123=500(valid)

3).Identifiers are case sensitive. Of course python language is case sensitive language.

Ex: total=500

TOTAL=500

Note:

-->If identifier starts with underscore(_) then it indicates it is private.

-->We can't use reserved words as identifiers **Ex: def=10**

-->There is no length limit for python identifiers. But not recommended to use too lengthy identifiers.

-->Dollar **($)** symbol is not allowed in python.

-->If identifier starts with **(_ _)**it indicates that strongly private identifiers.

-->If the identifier starts and ends with two underscores symbols then the identifier is language defined special name, which is also known as magic methods. **Ex: __add__**

D Y PATIL UNIVERSITY NAVI MUMBAI

Reserved Words: (Keywords)

->In python some words are reserved to represent some meaning or functionality. Such
type of words are called as Reserved words.
-->There are 30+ reserved words in python
-->True, False, None
-->and, or, not, is
-->if, else, elif
-->while, for, break, continue, return, in, yield
-->try, except, finally, raise, assert
-->import, from, as, class, def, global, nonlocal, lambda, del, with

Note:

-->All the reserved words in python contain only alphabets symbols.
-->Except the following 3 reserved words, all contain only lower case alphabets.
True, False, None

Ex: a=true(invalid)
a=True(valid)

```
>>> import keyword
>>> keyword . kwlist
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Data Types:

-->Any organization use cases or scenarios are 2-parts.

1).Data

2).Operations

-->Data of any organization use case can be represented in any programming language by using data types and variables.

-->Every programming language supports data types & variables, but the data type of any programming language are not going to be same with the data types & variables of other programming language.

-->Operations of any organization use case can be represented in any programming

language by using functions or methods.

-->Every programming language supports functions or methods or both.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Datatype:

-->Data types are nothing but some of the keywords of programming language, which are used to specify what type of data has to be store in the variable.

-->Without specifying data types variable memory allocation with not take place for

variables.

-->Without allocating memory space for the variable we can't store the data into variables.

-->Programming languages supports 2-types of data types.

1).Static

2).Dynamic

D Y PATIL UNIVERSITY
NAVI MUMBAI

1).Static Data Types:

-->In static type programming languages programmer should define the data type to the variable explicitly.

-->Once if we define data type to the variable explicitly, we can't modify the the data type of that variable through out the program execution.

-->In static DT supported languages, one variable can store only one verity of data.

-->C,C++,.NET, Java.....languages supports static data types.

2).Dynamic Data Types:

-->In dynamic data type supported languages, programmer should not define the data

types to the variables explicitly.

-->At the time of execution of program, data type of variable is decided based on the data which is assigned to the variable.

-->Dynamic DT supported languages, one variable can store different verities of data.

-->Python, java script.....languages are supported dynamic data types.

-->Every data type in python language is internally implemented as a class.

-->Python data types are categorized into 2-parts.

-->Fundamental types

-->Collection types

D Y PATIL UNIVERSITY
NAVI MUMBAI

inbuilt data types

Data Type	Description	Immutable	Example
int	We can use to represent the whole / integral values	Immutable	>>> a=10 >>> type(a)

Float	We can use to represent the decimal / floating point numbers	Immutable	>>> b=10.2
			>>> type(b)
			<class 'float'>

Complex	We can use to represent the complex numbers	Immutable	>>> c=10+5j
			>>> type(c)
			>>> c.real
			10.0
			>>> c.imag
			5.0

D Y PATIL UNIVERSITY
NAVI MUMBAI

inbuilt data types

Data Type	Description	Immutable	Example

Bool	WE can use to represent the logical values(Only allowed values are True and False)	Immutable	>>> flag=True
			>>> flag=False
			>>> type(flag) (a)
			<class 'bool'>

str	To represent sequence of characters	Immutable	>>> s='mahesh'
			>>> type(s)
			>>> s="mahesh"
			>>> s=''DYPATIL'''
			>>> type(s)

bytes	To represent a sequence of byte values from 0-256	Immutable	>>> list=[10,20,30,40]
			>>> b=bytes(list)
			>>> type(b)

D Y PATIL UNIVERSITY
NAVI MUMBAI

inbuilt data types

Data Type	Description	Immutable	Example
bytearray	To represent a sequence of byte values from 0-256	Muttable	>>>list=[10,20,30,40] >>> ba=bytearray(list) >>> type(ba)
range	To represent a range of values	Muttable	>>> r=range(10) >>> r1=range(10,20) >>> r2=range(10,50,5)
list	To represent an ordered collection of objects	Immutable	>>> l=[10,20,30,40] >>> type(l)

D Y PATIL UNIVERSITY NAVI MUMBAI

inbuilt data types

Data Type	Description	Immutable	Example
tuple	To represent an ordered collection of objects	Immutable	>>> t=(1,2,3,4,5) >>> type(t)
set	To represent an unordered collection of unique objects	Muttable	>>> s={1,2,3,4,5,6} >>> type(s)
list	To represent an ordered collection of objects	Immutable	>>> l=[10,20,30,40] >>> type(l)

D Y PATIL UNIVERSITY
NAVI MUMBAI

inbuilt data types

Data Type	Description	Immutable	Example
Frozenset	To represent an unordered collection of unique objects	Immutable	>>> s={1,2,3,'Mahesh', 100,'Durga’} >>> fs=frozenset(s) >>> type(fs)
dict	To represent a group of key value pair	Muttable	>>> d={100:'Mahesh',102:'Durga',103:'Sunny'} >>> type(d)

D Y PATIL UNIVERSITY
NAVI MUMBAI

```
a=10
id(a)
140712463448792

b=10
id(b)
140712463448792

a=10
id(a)
140712463448792
```

Python contains several inbuilt functions

1).**type():** To check the type of variable.

2).**id():** To get the address of object.

-->In python every thing is an object

3).**print():** To print the value

D Y PATIL UNIVERSITY
NAVI MUMBAI

int Data Type:

-->We can use int data type to represent whole numbers(Integral values).

Ex:

```
a=10
type(a)#int
```

Note:

-->In python-2 we have long data type to represent very large integer values.

-->But in python-3 there is no long type explicitly we can represent long values also by using int type only.

-->We can represent int values in the following ways.

1).Decimal form

2).Binary form

3).Octal form

4). Hexa decimal form

D Y PATIL UNIVERSITY NAVI MUMBAI

1).**Decimal Form(Base-10):**
-->It is the default number system in python. -->The allowed digits are :0 to 9.
Ex: a=10

---

2).Binary form(Base-2):
-->The allowed digits are : 0 & 1. -->Literal value should be prefixed with 0b or 0B.
Ex: a=0b1111(valid)
a=0B1111(valid)
a=0B123(Invalid)

3).Octal form(Base-8):
-->The allowed digits are :0 to 7 -->Literal value should be prefixed with 0o or 0O
Ex: a=0o123(Valid)
a=0O786(Invalid)

4).Hexa Decimal Form(Base-16):
-->The allowed digits are :0 to 9, a-f(Both lower and upper case are allowed). -->Literal value should be prefixed with 0x or 0X.
Ex: a=0XFACE(Valid)
a=0xBeef(Valid)
a=0XBeer(Invalid)

D Y PATIL UNIVERSITY
NAVI MUMBAI

Example

```
a=10

b=0b10

c=0o10

d=0x10

print(a)

10

print(b)

2
print(c)

8
print(d)

16
```


D Y PATIL
UNIVERSITY
NAVI MUMBAI

Float

-->We can use float data type to represent floating point values(decimal values).

```
>>> f=1.234
>>> type(f)
<class 'float'>
```

-->We can also represent floating point values by using exponential form(Scintific notation)

```
>>> f=1.2e3
>>> f
1200.0
```

Note: instead of 'e' we can use 'E'

Ex: f=1.2E3

-->The main advantage of exponential form is we can represent big values in less memory.

Note:

We can represent int values in decimal, binary,octal and hexa decimal forms. But we can represent float values by using decimal form only.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Complex Data Type

A complex is of the form
  a+bj
a-->Real part
b-->Imaginary part
j-->sqrt(-1)
-->a & b contains integers or floating point values.
 -->In the real part if we use int values then we can specify that either by decimal, octal, binary or hexa decimal form.
-->But imaginary part should be specified only by using decimal form.
Ex:
a=3+4j
a=1.5+2.5j
a=3+0B1010j(Invalid)
a=0b10101+3J(valid)
 -->We can perform operations on complex type values

```
>>> a=3+1.5j
>>> b=4+2.5j
>>> a+b
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Bool Data Type

>We can use this data type to represent Boolean values.
-->The only allowed values for this data type are:
True and False
-->Internally python represents **True as 1 and False as 0.**
Ex:
a=True
type(a)==>bool
Ex:
>>> a=10
>>> b=20
>>> c=a<b
>>> print(c)
True

>>>True+True==>2
>>>True-False==>1

D Y PATIL UNIVERSITY
NAVI MUMBAI

str Data Type

str represents String data type.

A string is a sequence of characters with in single quotes or double quotes.

Ex: s='Mahesh’

s="Mahesh"

-->By using single quotes or double quotes we can't represent multi line string literals.

Ex: s=“John

Dasari"(Invalid)

-->For this requirement we should go for triple quotes(''') or triple double quotes(""")

Ex: s=‘‘’John

Dasari'''

s="""“DY

Patil""" -->We can embeded one string in another string.

Ex: '''This "python classes very helpful" for java students'''.

D Y PATIL UNIVERSITY NAVI MUMBAI

Slicing of Strings

-->Slice means a piece.

-->[:] is called as slice operator, which can be used to retrieve parts of string.

-->In python string follows zero based index.

-->The index can be either +ve or -ve.

-->+ve index means forward direction from Left to Right.

-->-ve index means backward direction from Right to Left

D Y PATIL UNIVERSITY
NAVI MUMBAI

```
s="python"
s[0]
'p'
s[1]
'y'
s[-1]
'n'
s[30]
Traceback (most recent call last):
  File "<pyshell#59>", line 1, in <module>
    s[30]
IndexError: string index out of range
```

```
s[1:40]
'ython'
s[1:]
'ython'
s[:4]
'pyth'
s[:]
'python'
s[-1:4]
''
s[-4:-1]
'tho'
s[-4:-2]
'th'
s*3
'pythonpythonpython'
len(s)
6
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Note:

-->In the python the following data types are considered as fundamental data types.

-->int

-->float

-->complex

-->bool

-->str

-->In python , we can represent char values also by using str type and explicitly char type not available.

```
>>> c='a'
>>> type(c)
<class 'str'>
```

-->long data type is available in python2 but not in python3.

In python3 long values also we can represent by using int type only.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Type Casting

-->We can convert one type to other type. This conversion is called as Typecasting or Type coersion.

-->The following are various in-built functions for type casting.

-->int()

-->float()

-->complex()

-->bool()

-->str()

int(): We can use this function to convert values from other types to int.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Int type

We can use this function to convert values from other types to int.

```
int(123.456)
123

int(10+4j)
TypeError: int() argument must be a string, a bytes-like object or a real number, not 'comp

int(True)
1
int(False)
0
int("10")
10

int("10.5")
ValueError: invalid literal for int() with base 10: '10.5’

int("ten")
ValueError: invalid literal for int() with base 10: 'ten'
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Float

We can use float() to convert other type values to float type.

```
>>> float(10)
10.0
>>> float(10+4j)
TypeError: can't convert complex to float
>>> float(True)
1.0
>>> float(False)
0.0
>>> float("10")
10.0
>>> float("10.5")
10.5
>>>float("ten")
ValueError: could not convert string to float: 'ten'
```

Note:

-->We can convert any type value to float type expcept complex type.

-->Whenever we are trying to convert str type to float type compalsury str should be either integral or floating point literal and should be specified only in base-10.

D Y PATIL UNIVERSITY
NAVI MUMBAI

complex():

We can use complex() function to convert other data type to complex type.

Form-1:complex(x)

We can use this function to convert x into complex number with real part x and imaginary part 0.

```
>>> complex(10)
(10+0j)
>>> complex(10.5)
(10.5+0j)
>>> complex(True)
(1+0j)
>>> complex(False)
0j
>>> complex("10")
(10+0j)
>>> complex("10.5")
(10.5+0j)
>>> complex("ten")
ValueError: complex() arg is a malformed string
```


PATIL UNIVERSITY NAVI MUMBAI

Form-2:complex(x,y)

We can use this method to convert x and y into complex number such that x will be real part and y will be imaginary part.

```
>>> complex(10,2)
(10+2j)
>>> complex(10,-2)
(10-2j)
>>> complex(True,False)
(1+0j)
>>> complex("10","10.2")
TypeError: complex() can't take second arg if first is a string
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

bool()

We can use this function to convert other type to bool type

```
>>> bool(0)
False
>>> bool(1)
True
>>> bool(10)
True
>>> bool(0.123)
True
>>> bool(10.5)
True
>>> bool(0.0)
False
>>> bool(10+4J)
True
>>> bool(0+0j)
False
>>> bool("True")
True
>>> bool("")
False
>>> bool(None)
False
>>> bool(" ")
True
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

str():

We can use this method to convert other type values to str type.

```
>>> str(10)
'10'
>>> str(10.5)
'10.5'
>>> str(True)
'True'
>>> str(10+4j)
'(10+4j)'
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Fundamental Data types vs Immutability

-->All the fundamental data types are immutable. i.e once we creates an object, we can't perform any changes in that object. If we are trying to change then with those changes a new object will be created. This non-changable behaviour is called as immutability.

-->In python if a new object is required, then PVM won't create object immediately. First it will check if any object is available with the required content or not. If available then existing object will be re-used. If it is not available then only a new object will be created. The advantage of this approach is memory utilization and performance will be improved.
-->But the problem in this approach is, several references pointing to the same object, by using one reference if we are allowed to change the content in the existing object then the remaining references will be effected. To prevent this immutability concept is required.

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Accordint to this once we create an object we are not allowed to change content. If we are trying with those changes a new object will be created.

```
>>> a=10
>>> b=10
>>> id(a)
140723035886912
>>> id(b)
140723035886912
>>> a is b
True
```

```
>>> a=10+5j
>>> b=10+5j
>>> id(a)
2560497332016
>>> id(b)
2560497331568
>>> a is b
False
```

No-Reusability

```
>>> a=True
>>> b=True
>>> a is b
True
>>> id(a)
140723035355472
>>> id(b)
140723035355472
```

```
>>> a='mahesh'
>>> b='mahesh'
>>> id(a)
3003052190160
>>> id(b)
3003052190160
>>> a is b
True
```

```
>>> a=10.123
>>> b=10.123
>>> id(a)
3003011087360
>>> id(b)
3003011087408
>>> a is b
False
```

No-Reusability

Note:

-->There is no Re-Usability for float & complex types.
-->Re-Usability is applicable for int only the range 0-256.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Variables

Variables are containers for storing data values.

Creating Variables

Python has no command for declaring a variable.

A variable is created the moment you first assign a value to it.

example:

```
x = 5
y = "John"
print(x)
print(y)
```

Variables do not need to be declared with any particular *type*, and can even change type after they have been set.

```
x = 4       # x is of type int
x = "Sally" # x is now of type str
print(x)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Variable Names

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)
- A variable name cannot be any of the Python keywords.

Example:

Legal variable names:

```
myvar = "John"
my_var = "John"
_my_var = "John"
myVar = "John"
MYVAR = "John"
myvar2 = "John"
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Multi Words Variable Names

Variable names with more than one word can be difficult to read.

There are several techniques you can use to make them more readable:

Camel Case

Each word, except the first, starts with a capital letter:

myVariableName = "John"

Pascal Case

Each word starts with a capital letter:

MyVariableName = "John"

Snake Case

Each word is separated by an underscore character:

my_variable_name = "John"

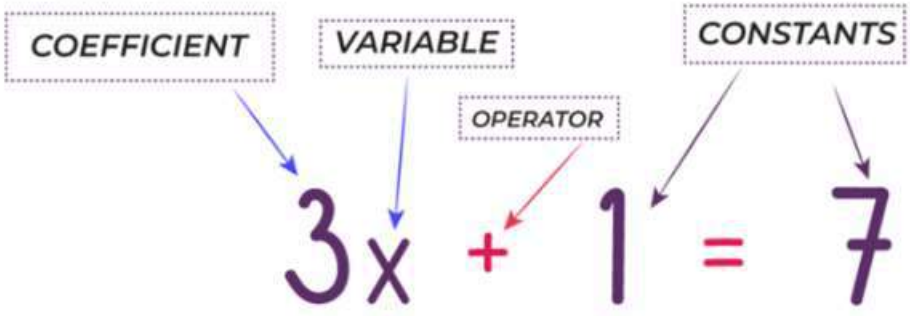
D Y PATIL UNIVERSITY
NAVI MUMBAI

Literals in Python

- **Literals** in Python are fixed values written directly in the code that represent constant data.
- They provide a way to store numbers, text, or other essential information that does not change during program execution.
- Python supports different types of literals, such as numeric literals, string literals, Boolean literals, and special values like None.

For example:

- 10, 3.14, and 5 + 2j are numeric literals.
- 'Hello' and "Python" are string literals.
- True and False are Boolean literals.



D Y PATIL UNIVERSITY
NAVI MUMBAI

Numeric Literals

Numeric literals represent numbers and are classified into three types:

- **Integer Literals** – Whole numbers (positive, negative, or zero) without a decimal point. **Example**: 10, -25, 0
- **Floating-point (Decimal) Literals** – Numbers with a decimal point, representing real numbers. **Example**: 3.14, -0.01, 2.0
- **Complex Number Literals** – Numbers in the form a + bj, where a is the real part and b is the imaginary part. **Example**: 5 + 2j, 7 - 3j

```
# Integer literals
a = 100
b = -50

# Floating-point literals
c = 3.14
d = -0.005

# Complex number literals
e = 4 + 7j
f = -3j

print(a, b, c, d, e, f)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

String Literals

String literals are sequences of characters enclosed in quotes. They are used to represent text in Python.

Types of String Literals:

- **Single-quoted strings** – Enclosed in single quotes (' '). Example: 'Hello, World!'
- **Double-quoted strings** – Enclosed in double quotes (" "). Example: "Python is fun!"
- **Triple-quoted strings** – Enclosed in triple single (''' ''') or triple double (""" """) quotes, generally used for multi-line strings or docstrings. Example:

```
'''This is
a multi-line
string'''
```

- **Raw strings** – Prefix with r to ignore escape sequences (\n, \t, etc.). Example: r"C:\Users\Python" (backslashes are treated as normal characters).

D Y PATIL UNIVERSITY
NAVI MUMBAI

Example

```
# Different string literals
a = 'Hello'      # Single-quoted
b = "Python"     # Double-quoted
c = '''This is
a multi-line string'''  # Triple-quoted
d = r"C:\Users\Python"  # Raw string

print(a)
print(b)
print(c)
print(d)
```

Output

```
Hello
Python
This is a multi-line string
C:\Users\Python
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Boolean Literals

Boolean literals represent truth values in Python. They help in decision-making and logical operations. Boolean literals are useful for controlling program flow in conditional statements like if, while, and for loops.

Types of Boolean Literals:

- True – Represents a positive condition (equivalent to 1).
- False – Represents a negative condition (equivalent to 0).

```
# Boolean literals
a = True
b = False

print(a, b)       # Output: True False
print(1 == True)  # Output: True
print(0 == False) # Output: True
print(True + 5)   # Output: 6 (1 + 5)
print(False + 7)  # Output: 7 (0 + 7)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Basic input output and command-line input

- Understanding input and output operations is fundamental to Python programming.
- With the print() function, we can display output in various formats, while the input() function enables interaction with users by gathering input during program execution.
- **Reading dynamic input from the keyboard:**
- In python-2 the following 2-functions are available to read dynamic input from the keyboard.

  1).raw_input()

  2).input()

D Y PATIL UNIVERSITY
NAVI MUMBAI

1).raw_input():

This function always reads the data from the keyboard in the form of string format. We have to convert that string type to our required type by using the corresponding type casting methods.

```
1) x=raw_input("Enter some value:")
2) print(type(x))==>It always print str type only for any input type
```

2).input():

input() function can be used to read data directly in our required format. We rae not required to perform type casting.

Ex:

```
1) x=input(Enter value")
2) type(x)
3) 10==>int
4) "mahesh"==>str
5) 10.5==>float
6) True==>bool
```

Note:

-->But in python-3 we have only input() method and raw_input() method is not available.

-->Python-3 input() function behaviour exactly same as raw_input() method in python-2. ie every input value is treated as str type only.

-->raw_input() function of python-2 is renamed as input() function in python-3.

D Y PATIL UNIVERSITY NAVI MUMBAI

```
>>> name=input("Enter your name:");
    Enter your name:John
>>> print("Hello",name,"Welcome!");
    Hello John Welcome!
>>>
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Form-1: print() without argument

just it prints new line character.

Form-2: print(String)

print("Hello world")-->We can use escape characters also

```
print("Hello \n world")
print("Hello \t world")
```

-->We can use repetetion operator(*) in the string

```
print(10*"hello")
print("hello"*10)
```

-->We can use + operator also

```
print("Hello" + "World")
```

Note:

-->If both arguments are string type then + operator acts as concatination operator.

-->If one argument is string type and second is any other type like int then we eill get an error.

-->If both arguments are number type then + operator avts as arithmetic addition operator.

Q: what is diff between

print("Hello"+"world")==>Helloworld

print("Hello","world")==>Hello word

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Form-3: print() with variable number of arguments

Ex:

```
1) a,b,c=10,20,30
2) print("The values are: ",a,b,c)
```

o/p: The values are: 10 20 30

-->Bydefault output values are separated by space. If we want we can specify separator by using "sep" attribute.

Ex:

```
1) a,b,c=10,20,30
2) print(a,b,c,sep=',')
3) print(a,b,c,sep=':')
```

o/p:D:\pythonclasses>py test.py

10,20,30

10:20:30

D Y PATIL UNIVERSITY NAVI MUMBAI

Form-4:print() with end attribute

```
1) print("Hello")
2) print("Durga")
3) print("Soft")
```

o/p: Hello
Durga
Soft

-->If we want output in the same line with space

```
1) print("Hello",end=' ')
2) print("Durga",end=' ')
3) print("Soft")
```

o/p:D:\pythonclasses>py test.py
Hello Durga Soft

Note: The default value of end attribute is \n, which is nothing but new line character.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Form-5:print(object) statement

-->We can pass any object(like list, tuple, set etc) as argument to the print statement.

Ex:

```
1) l=[10,20,30,40]
2) t=(10,20,30,40)
3) print(l)
4) print(t)
```

Form-6:print(String, variable list):

-->We can use print() statement with string and any number of arguments.

Ex:

```
1) s="Mahesh"
2) a=50
3) s1="Selenium"
4) s2="Python"
5) print("Hello",s,"Your age is :",a)
6) print("You are teaching", s1,"and", s2)
```

o/p: Hello Mahesh Your age is : 50
You are teaching Selenium and Python

D Y PATIL UNIVERSITY
NAVI MUMBAI

Form-7:print(formatted string):

%i====>int

%d===>int

%f===>float

%s===>string type

syn: print("formatted string" %(variable list))

Ex:

```
1) a=10
2) b=20
3) c=30
4) print("a value is %d" %a)
5) print("b value is %d and c value is %d" %(b,c))
```

o/p: a value is 10

b value is 20 and c value is 30

D Y PATIL UNIVERSITY NAVI MUMBAI

Command Line Arguments

-->argv is not an array it is a list. It is available in sys module.
-->The arguments which are passing at the time of execution are called as command line arguments.
ex: D:\pythonclasses>py test.py 10 20 30
-->Within the python program this command line arguments are available in argv. which is present in sys module.

Note: argv[0] represents name of the program, but not the first command line argument. argv[1] represents First command line argument.

Program: To check type of argv from sys.

```
1) from sys import argv
2) print(type(argv))
```

o/p: D:\pythonclasses>py test.py 10 20
<class 'list'>

D Y PATIL UNIVERSITY
NAVI MUMBAI

Example

```
import sys
print("Total arguments:", len(sys.argv))
print("Script name:", sys.argv[0])
print("Arguments:", sys.argv[1:])
total = 0
for arg in sys.argv[1:]:
    total += int(arg)
print("Sum =", total)
```

Output:

```
Total arguments: 3
Script name: C:/Swati/pythonCode/add.py
Arguments: ['10', '20']
Sum = 30
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Python Operator:

In Python programming, Operators in general are used to perform operations on values and variables.

Operators: Special symbols like -, + , * , /, etc.

Operands: Value on which the operator is applied.

Types of Operators in Python

Operators in Python

Type	Operators
Arithmetic Operators	+, -, *, /, %
Relational Operators	< , <= , > , >=, ==, !=
Logical Operators	AND, OR, NOT
Bitwise operator	&, |, ^, ~, <<, >>
Assignment Operator	= , += , -= , *= , %=

D Y PATIL UNIVERSITY
NAVI MUMBAI

Arithmetic Operator

Used to perform mathematical operations.

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus	a % b
**	Exponent	a ** b
//	Floor Division	a // b

D Y PATIL UNIVERSITY
NAVI MUMBAI

Example

```
a = 10
b = 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
print(a // b)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Comparison (Relational) Operators

Used to compare two values and return True or False.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Example:

```
a = 10
b = 3
print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Logical Operators

Operator	Description
and	True if both conditions are True
or	True if any one condition is True
not	Reverses result

Example:

```
a = 10
b = 5
print(a > 5 and b < 10)   # True
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Assignment Operators

Used to assign values to variables.

Operator	Example	Meaning
=	a = 10	Assign
+=	a += 5	a = a + 5
-=	a -= 5	a = a - 5
*=	a *= 2	a = a * 2
/=	a /= 2	a = a / 2

D Y PATIL UNIVERSITY
NAVI MUMBAI

Bitwise Operators

Used to perform operations at **bit level**.

Operator	Meaning
&	AND
`	`
^	XOR
~	NOT
<<	Left Shift
>>	Right Shift

Example:

```
a = 5   # 0101
b = 3   # 0011
print(a & b)  # 1
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Membership Operators

We can use membership operators to check whether the given object present in the given collection.(Ot may be string, list, tuple, set or Dict).

in==>Returns True if the given object present in the specified collection.

not in==>Returns True if the given object not present in the specified collection.

Operator	Meaning
in	Value present
not in	Value not present

Example:

```
x = "Python"
print("P" in x)      # True
print("z" not in x)  # True
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Identity Operators

Operator	Meaning
is	Same object
is not	Different object

Output:

```
a = 10
b = 10
print(a is b)   # True
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Special Operators

Ternary Operator

```
result = "Even" if a % 2 == 0 else "Odd"
```

Operator Precedence

Defines order of execution.

Highest → Lowest:

1.**

2.* / % //

3.+ -

4.== != > <

5.and

6.or

D Y PATIL UNIVERSITY
NAVI MUMBAI

Defines order of execution.

Highest → Lowest:

1.**

2.* / % //

3.+ -

4.== != > <

5.and

6.or

D Y PATIL UNIVERSITY
NAVI MUMBAI

Python Comments

- Comments in Python are the lines in the code that are ignored by the interpreter during the execution of the program.
- It enhance the readability of the code.
- It can be used to identify functionality or structure the code-base.
- It can help understanding unusual or tricky scenarios handled by the code to prevent accidental removal or changes.
- It can be used to prevent executing any specific part of your code, while making changes or testing.

```
# I am single line comment

""" Multi-line comment used
print("Python Comments") """
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Types of Comments in Python

1. Single-Line Comment

- Starts with `#`
- Everything after `#` is treated as a comment

Example:

```python
# This is a single-line comment
x = 10  # Variable assignment
```

2. Multi-Line Comment

Python does not have a separate syntax for multi-line comments.

But we use **triple quotes** (`'''` or `"""`) for multi-line comments or documentation.

Example:

```python
'''
This is a multi-line comment
Used to explain large blocks of code
'''
print("Hello Python")
```

or

```python
"""
This is also a multi-line comment
"""
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Indentation in Python

In Python, indentation is used to define blocks of code.

It tells the Python interpreter that a group of statements belongs to a specific block. All statements with the same level of indentation are considered part of the same block.

Indentation is achieved using whitespace (spaces or tabs) at the beginning of each line.

```
if 10 > 5:
    print("This is true!")
    print("I am tab indentation")

print("I have no indentation")
```

Output

```
This is true!
I am tab indentation
I have no indentation
```

- The first two print statements are indented by 4 spaces, so they belong to the if block.
- The third print statement is not indented, so it is outside the if block.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Indentation Rules in Python

1. **Indentation is compulsory**
2. Use **same indentation level** for statements in the same block
3. Recommended indentation = **4 spaces**
4. Mixing **tabs and spaces** is not allowed
5. Indentation starts after : (colon)

D Y PATIL UNIVERSITY
NAVI MUMBAI

Indentation in Conditional Statements

if Statement

```python
if a > b:
    print("a is greater")
```

if-else Statement

```python
if a > b:
    print("a is greater")
else:
    print("b is greater")
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Indentation in Loops

for Loop

```python
for i in range(3):
    print(i)
```

while Loop

```python
i = 1
while i <= 3:
    print(i)
    i += 1
```

Nested Indentation

```python
if x > 0:
    if x % 2 == 0:
        print("Positive Even")
    else:
        print("Positive Odd")
```

Each nested block increases indentation level.

Indentation in Functions

```python
def greet():
    print("Hello")
    print("Welcome to Python")
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Indentation is Incorrect

```
if (condition){
print("...")
elif (condition){
print("...")
else{
print("...")
```

Indentation is Correct

```
if (condition){
    print("...")
elif (condition){
    print("...")
else{
    print("...")
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Creating Your First Application in Python

Step 1: Your First Python Program

Let's start with a simple application that prints a welcome message by following the steps below:

- Open any text editor you like (Notepad, VSCode, PyCharm, etc.) and write the following Python code:

print("Welcome to D Y Patil University!")

- Save the file as **welcome.py** (the .py extension tells the system it's a Python file).
- *Open your terminal or command prompt and navigate to the folder where gfg.py is saved and run the program by typing:*

***python* welcome.py**

4. This will result in Python executing the code in the *gfg.py* file as shown be

```
C:\Swati\pythonCode>python welcome.py
Welcome to D Y Patil University!

C:\Swati\pythonCode>
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Conditional Statements in Python

Conditional statements in Python are used to execute certain blocks of code based on specific conditions.

These statements help control the flow of a program, making it behave differently in different situations.

Types of Conditional Statements:

1.If Conditional Statement

2.If else Conditional Statement

3.elif Statement

4.Nested if..else Conditional Statement

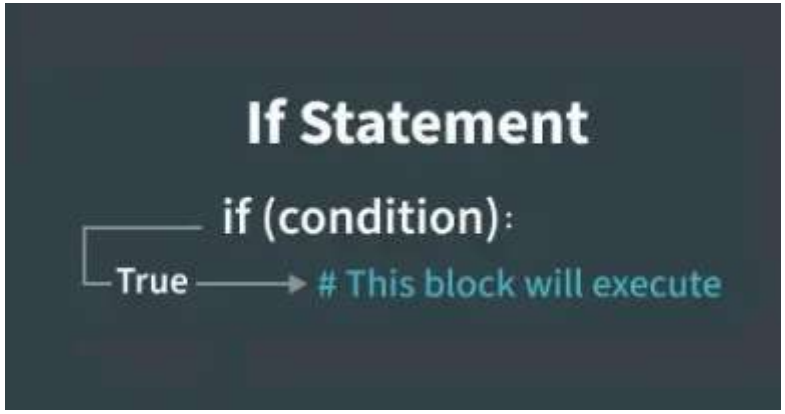
5.Ternary Conditional Statement

6.Match-Case Statement

D Y PATIL UNIVERSITY
NAVI MUMBAI

If Conditional Statement

- If statement is the simplest form of a conditional statement.
- It executes a block of code if the given condition is true.




```python
age = 20
if age >= 18:
    print("Eligible to vote.")
```

Output

```
Eligible to vote.
```

Short Hand if

Short-hand if statement allows us to write a single-line if statement.

```python
age = 19
if age > 18: print("Eligible to Vote.")
```

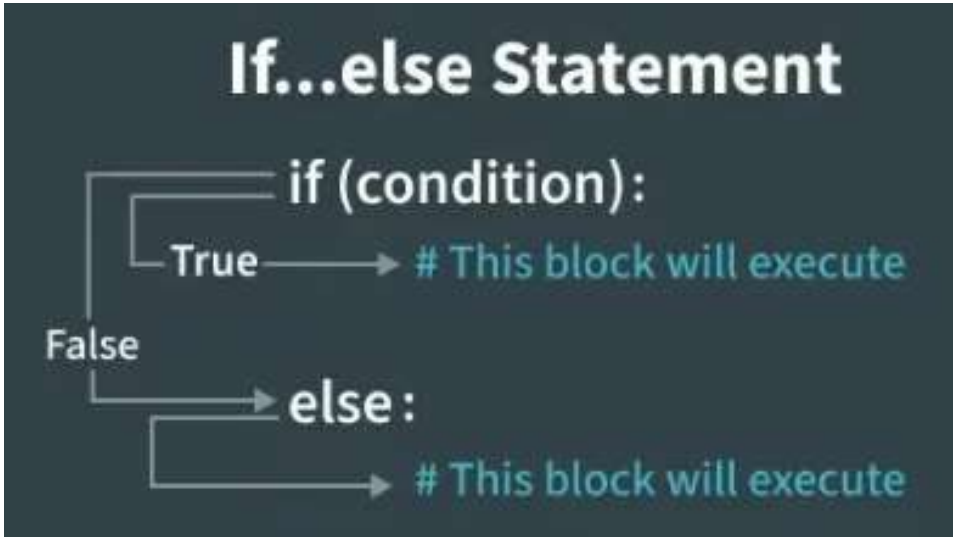
Output

```
Eligible to Vote.
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

If else Conditional Statement

- If Else allows us to specify a block of code that will execute if the condition(s) associated with an if or elif statement evaluates to False.
- Else block provides a way to handle all other cases that don't meet the specified conditions.



D Y PATIL UNIVERSITY
NAVI MUMBAI

```python
age = 10
if age <= 12:
    print("Travel for free.")
else:
    print("Pay for ticket.")
```

Output

```
Travel for free.
```

Short Hand if-else

The short-hand if-else statement allows us to write a single-line if-else statement.

```python
marks = 45
res = "Pass" if marks >= 40 else "Fail"
print(f"Result: {res}")
```

Output

```
Result: Pass
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Interactive Application (Odd or Even Number Checker)

Now, let's make your program interactive. This time, it will determine whether a given number is odd or even by following the steps below:

1. First, we need to get a number from the user. In Python, use the input() function, and convert the input string to an integer using int():

```
num = int(input("Enter a number: "))
```

2. Next, use a conditional statement to check if the number is divisible by 2:

```
if num % 2 == 0:
    print("It's an Even number!")
else:
    print("It's an Odd number!")
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

3. Combine these into one script and save it as EvenOdd.py:

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print("It's an Even number!")
else:
    print("It’s an Odd number!")
```

4. Run it again via the command line:

python EvenOdd.py

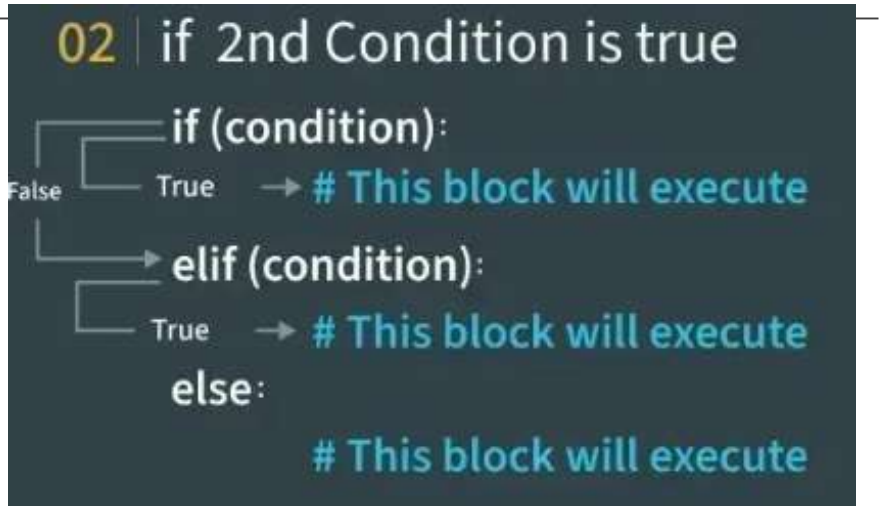
```
==================== RESTART: C:\Swati\pythonCode\EvenOdd.py =
Enter a number: 22
It's an Even number!

==================== RESTART: C:\Swati\pythonCode\EvenOdd.py =
Enter a number: 25
It's an Odd number!
```

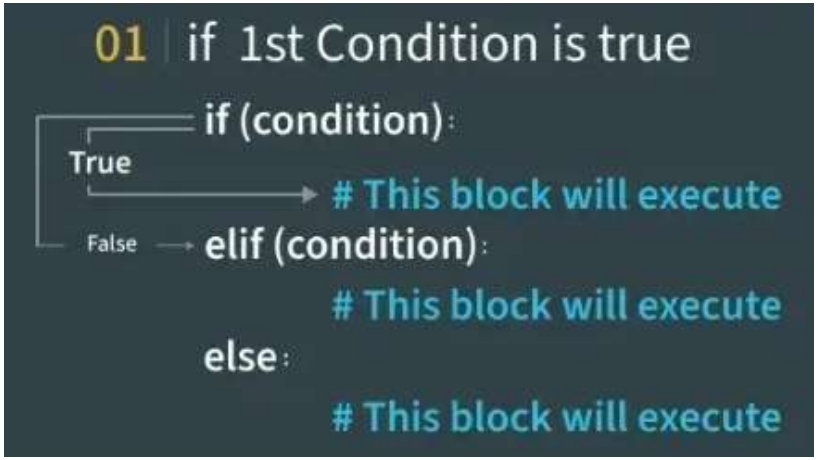
Note: This method is also known as ternary operator. Ternary Operator essentially a shorthand for the if-else statement that allows us to write more compact and readable code, especially for simple conditions.

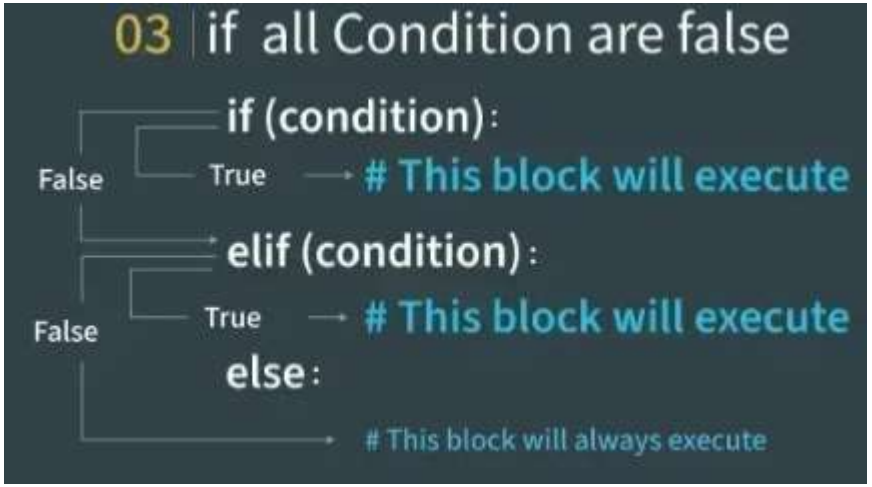
D Y PATIL UNIVERSITY NAVI MUMBAI

elif Statement

- elif statement in Python stands for "else if."
- It allows us to check multiple conditions, providing a way to execute different blocks of code based on which condition is true.
- Using elif statements makes our code more readable and efficient by eliminating the need for multiple nested if statements.







Example:

```
age = 25
if age <= 12:
    print("Child.")
elif age <= 19:
    print("Teenager.")
elif age <= 35:
    print("Young adult.")
else:
    print("Adult.")
```

Output

Young adult.

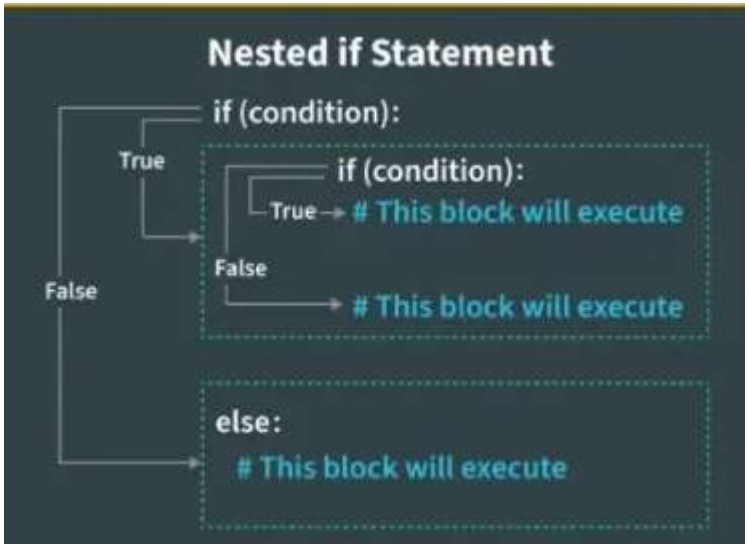
The code checks the value of age using if-elif-else. Since age is 25, it skips the first two conditions (age <= 12 and age <= 19), and the third condition (age <= 35) is True, so it prints "Young adult.".

D Y PATIL UNIVERSITY
NAVI MUMBAI

Nested if..else Conditional Statement

Nested if..else means an if-else statement inside another if statement.

We can use nested if statements to check conditions within conditions.




```
age = 70
is_member = True

if age >= 60:
    if is_member:
        print("30% senior discount!")
    else:
        print("20% senior discount.")
else:
    print("Not eligible for a senior discount.")
```

Output

30% senior discount!

D Y PATIL UNIVERSITY
NAVI MUMBAI

Ternary Conditional Statement

A ternary conditional statement is a compact way to write an if-else conditio

sometimes called a "conditional expression."

```
# Assign a value based on a condition
age = 20
s = "Adult" if age >= 18 else "Minor"
print(s)
```

Output

```
Adult
```

Here:

- If age >= 18 is True, status is assigned "Adult".
- Otherwise, status is assigned "Minor".

Python Match Case Statement

Introduced in Python 3.10, the match case statement offers a powerful mechanism for pattern matching in Python.

It allows us to perform more expressive and readable conditional checks.

Unlike traditional if-elif-else chains, which can become unwieldy with complex conditions, the match-case statement provides a more elegant and flexible solution.

```
match subject:
    case pattern1:
        # Code block if pattern1 matches
    case pattern2:
        # Code block if pattern2 matches
    case _:
        # Default case (wildcard) if no other pattern matches
```

- **match subject:** The value (or variable) to match against.
- **case pattern:** A pattern to match the subject.
- **_ (Wildcard):** A default catch-all pattern, similar to a "default" in other languages' switch statements.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Example

```
def check_number(x):
    match x:
        case 10:
            print("It's 10")
        case 20:
            print("It's 20")
        case _:
            print("It's neither 10 nor 20")

check_number(10)
check_number(30)
```

Output:

It's 10

It's neither 10 nor 20

D Y PATIL UNIVERSITY
NAVI MUMBAI

Match Case Statements with Constants

Matching constants is one of the simplest uses of the match-case statement. It checks if a variable matches specific constant values and allows for different actions based on the matched constant.

```
def greet(person):
    match person:
        case "A":
            print("Hello, A!")
        case "B":
            print("Hello, B!")
        case _:
            print("Hello, stranger!")

greet("A")
greet("B")
```

Explanation:

- The function greet(person) checks the value of person using the match-case statement.
- It specifically matches "A" and "B", printing a personalized greeting.
- If the value of person doesn't match any of the constants, the wildcard _ matches any value and prints "Hello, stranger!".

D Y PATIL UNIVERSITY
NAVI MUMBAI

Iterative Statements

-->If we want to execute a group of statements multiple times then we should go for iterative statements.

-->Python supports 2-types of iterative statements.

1). for loop

2).while loop

D Y PATIL UNIVERSITY
NAVI MUMBAI

for loop:

-->If we want to execute some action for every element present in some sequence(IT may
be string or collection) then we should go for for loop.

Syntax:

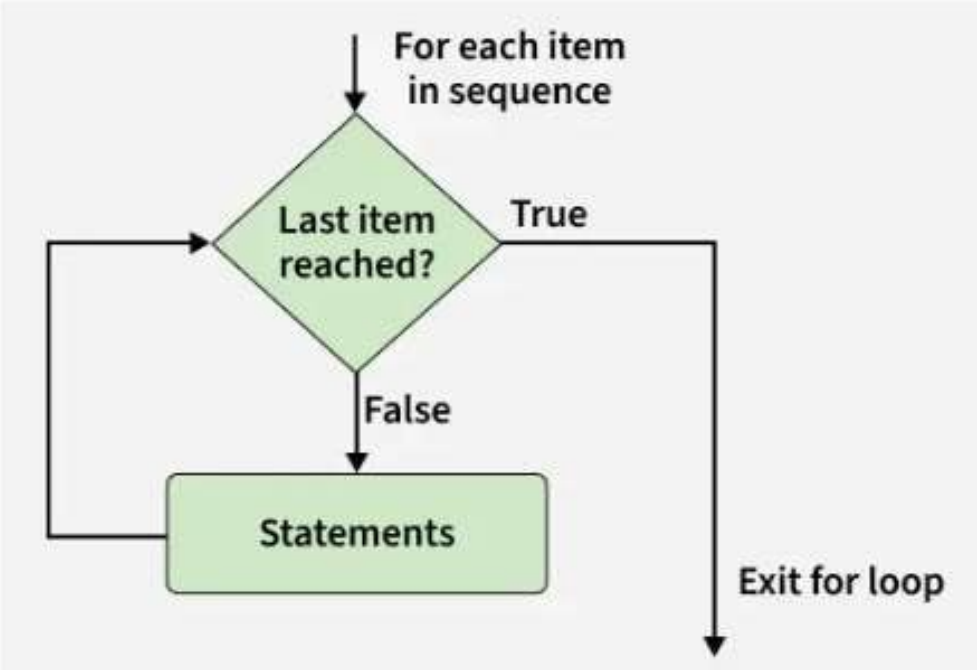
```
for x in sequence:
    Body
```

Ex: To print the characters present in the given string.

```
s="John"
for x in s:
    print(x)
```

Output:

J
o
h
n



D Y PATIL
UNIVERSITY
NAVI MUMBAI

Example

```
s="John"
i=0
for x in s:
    print("The character present at",i,"index",x)
    i=i+1
```

Output:

```
The character present at 0 index J
The character present at 1 index o
The character present at 2 index h
The character present at 3 index n
```

Ex: To print Hello 10 times.

```
for x in range(10):
    print("Hello")
```

Ex:To display numbers from 0-10

```
for x in range(10):
    print(x)
```

Ex:To display odd numbers from 0-20

```
for x in range(21):
    if(x%2!=0):
        print(x)
```

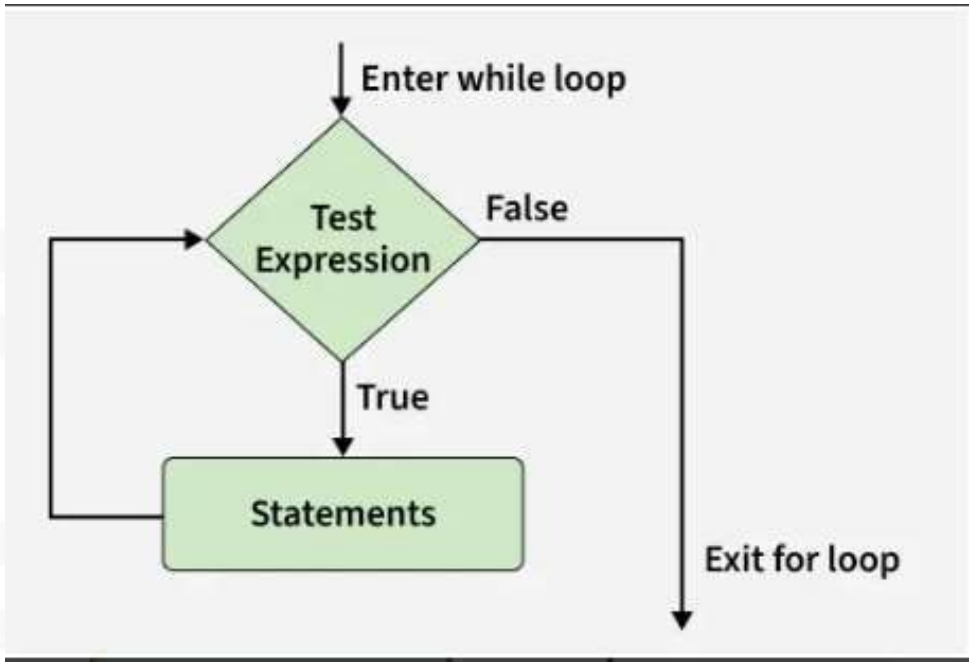

D Y PATIL UNIVERSITY
NAVI MUMBAI

while loop:

-->If we want to execute a group of statements iteratively until some condition false, then we should go for while loop.

Syn:

```
while condition:
        body
```



Ex: To print numbers from 1 to 10 by using while loop

```
x=1
while x<=10:
    print(x)
    x+=1
```

Ex: To display the sum of fisrt n numbers

```
n=int(input("Enter Number:"))
sum=0
i=1
while i<=n:
  sum=sum+i
  i+=1
print("The sum of the first",n,"numbers is:",sum)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Ex: w.a.p to prompt user to enter some name until entering mahesh.

```
1) name=""
2) while name!="mahesh":
3)     name=input("Enter Name:")
4) print("Thanks for confirmation")
```

Ex:w.a.p to prompt user to enter name & pwd until mahesh & python

```
1) username=""
2) password=""
3) while (username!="mahesh") or (password!="python"):
4)     username=input("Enter correct username :")
5)     password=input("Enter correct pwd:")
6) print("Thanks for confirmation")
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Infinite While Loop

If we want a block of code to execute infinite number of times then we can use the while loop in Python to do so.

Code given below uses a 'while' loop with the condition "True", which means that the loop will run infinitely until we break out of it using "break" keyword or some other logic.

Example:

```
while (True):
    print("Hello Geek")
```

Note: *It is suggested not to use this type of loop as it is a never-ending infinite loop where the condition is always true and we have to forcefully terminate the compiler.*

D Y PATIL UNIVERSITY
NAVI MUMBAI

Nested Loop

-->Sometimes we can take a loop inside anopther loop, which are also known as nested loops

Ex:

```
1) for i in range(5):
2)   for j in range(4):
3)     print("i=",i," j=",j)
```

Alternative way:

```
1) for i in range(5):
2)   for j in range(4):
3)     print("i={} and j={}".format(i,j))
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

```
num=int(input("Enter the number:"));
for i in range(1, num+1):
    for j in range(i):
        print(i, end=' ')
    print()
```

output:

```
Enter the number:5
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

1. Continue Statement:

The continue statement in Python returns the control to the beginning of the loop.

```
for letter in 'Welcome':
    if letter == 'e' or letter == 'c':
        continue
    print('Current Letter :', letter)
```

Output:

```
==========================
Current Letter : W
Current Letter : l
Current Letter : o
Current Letter : m
```

Explanation: The continue statement is used to skip the current iteration of a loop and move to the next iteration. It is useful when we want to bypass certain conditions without terminating the loop.

D Y PATIL UNIVERSITY NAVI MUMBAI

Break Statement

The break statement in Python brings control out of the loop.

```
for letter in 'Welcome':
    if letter == 'l' or letter == 'c':
        break
    print('Current Letter :', letter)
```

Output:

```
Current Letter : W
Current Letter : e
```

Explanation: break statement is used to exit the loop prematurely when a specified condition is met. In this example, the loop breaks when the letter is either 'e' or 's', stopping further iteration.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Pass Statement

We use pass statement in Python to write empty loops. Pass is also used for empty control statements, functions and classes.

```
for letter in 'welcome':
    pass
print('Last Letter :', letter)
```

Output:

Last Letter : e

Explanation: In this example, the loop iterates over each letter in 'geeksforgeeks' but doesn't perform any operation, and after the loop finishes, the last letter ('s') is printed.

D Y PATIL UNIVERSITY
NAVI MUMBAI

String

The most commonly used object in any project and in any programming language is String only. Hence we should aware complete information about the string DT.

What is a String?

Any sequence of characters within either single quotes or double quotes is considered as a String.

Syn:

```
s='mahesh'
s="mahesh"
```

Note:

In most of other languages like C, C++, java a single character with in single quotes is treated as char data type value. But in python we are not having char data type. Hence it is treated as String only.

Ex:

```
>>> ch='a'
>>> type(ch)
  <class 'str'>
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

How to define multiline string literals:

We can define multi-line string literals by using triple single or double quotes.

Ex:

```
>>>s=‘’’Info
      Software
          Solutions'’’
```

-->We can also use triple quotes to use single quotes or double quotes as symbol inside the string.

Ex:

1) s='This is ' a single quote symbol'==>(Invalid)
2) s='This is \' a single quote symbol'==>(valid)
3) s="This is ' a single quote symbol"==>(valid)
4) s='This is " a double quote symbol'==>(valid)
5) s='The "Python Notes" by 'mahesh' is very helpful'==>(Invalid)
6) s="The "Python Notes" by 'mahesh' is very helpful"==>(Invalid)
7) s='The \"Python Notes\" by \'mahesh\' is very helpful'==>(valid)
8) s='''The "Python Notes" by 'mahesh' is very helpful'''==>(valid)

D Y PATIL UNIVERSITY
NAVI MUMBAI

How to access characters of a String:

We can access characters of a string by using the following ways.
1).By using Index
2).By using slice operator

1).By using Index:

- Python supports both +ve and -ve index.
- +ve index means left to right(Forward Direction)
- -ve means right to left(Backward Direction)

Ex:

```
      -6- 5  -4 -3 -2-1
 s = 'm  a  h  e  s  h'
       0   1  2  3 4  5
```

Ex:
1) s[0]==>m
2) s[-1]==>h
3) s[10]==>String index out of range.

D Y PATIL UNIVERSITY
NAVI MUMBAI

w.ap to accept some string from the keyboard and display its characters by index wise(both +ve and –ve)

Example:

```
 s=input("Enter some string:")
i=0
for x in s:
    print("The character present at Positive index{} and Negative Index{} is{}".format(i,i-len(s),x))
    i=i+1
```

Output:

```
Enter some string:welcome
The character present at Positive index0 and Negative Index-7 isw
The character present at Positive index1 and Negative Index-6 ise
The character present at Positive index2 and Negative Index-5 isl
The character present at Positive index3 and Negative Index-4 isc
The character present at Positive index4 and Negative Index-3 iso
The character present at Positive index5 and Negative Index-2 ism
The character present at Positive index6 and Negative Index-1 ise
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

String Iteration

Strings are iterable; you can loop through characters one by one.

Example: Here, it print each character on its own line.

```
s = "Python"
for char in s:
    print(char)
```

Output

```
P
y
t
h
o
n
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

String Immutability

Strings are **immutable,** which means that they cannot be changed after they are created. If we need to manipulate strings then we can use methods like **concatenation, slicing** or **formatting** to create new strings based on original.

Example: In this example we are changing first character by building a new string.

```
s = "welcomeTODyPatil"
s = "W" + s[1:]   # create new string
print(s)
```

Output:

WelcomeTODyPatil

D Y PATIL UNIVERSITY
NAVI MUMBAI

Deleting a String

In Python, it is not possible to delete individual characters from a string since strings are immutable. However, we can delete an entire string variable using the del keyword.

Example: Here, we are using del keyword to delete a string.

```
s = "GfG"
del s
```

Note: *After deleting the string if we try to access s then it will result in a* ***NameError*** *because variable no longer exists.*

Updating a String:

As strings are immutable, "updates" create new strings using slicing or methods such as replace().

Example: This code fix the first letter and replace a word.

```
s = "hello Jimmy"
s1 = "H" + s[1:]                # update first
character
s2 = s.replace("Jimmy", "John")  # replace
word
print(s1)
print(s2)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Common String Methods

Python provides various built-in methods to manipulate strings. Below are some of the most useful methods:

1. **len()**: The len() function returns the total number of characters in a string (including spaces and punctuation).

```
s = "WelcomeTODyPatil"
print(len(s))
```

Output:

16

2. **upper() and lower():** upper() method converts all characters to uppercase whereas, lower() method converts all characters to lowercase.

Example:

```
s = "Hello World"
print(s.upper())
print(s.lower())
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

3) strip() and replace(): strip() removes leading and trailing whitespace from the string and replace() replaces all occurrences of a specified substring with another.

Example:

```
s = "   Gfg   "
print(s.strip())
s = "Python is fun"
print(s.replace("fun", "awesome"))
```

Output

Gfg

Python is awesome

D Y PATIL UNIVERSITY
NAVI MUMBAI

Concatenating and Repeating Strings

We can concatenate strings using + operator and repeat them using * operator.

1. Strings can be combined by using **+ operator**.

Example: Join two words with a space.

```
s1 = "Hello"
s2 = "World"
print(s1 + " " + s2)
```

Output

```
Hello World
```

2. We can repeat a string multiple times using *** operator**.

Example: Repeat a greeting three times.

```
s = "Hello "
print(s * 3)
```

Output

```
Hello Hello Hello
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Formatting Strings

Python provides several ways to include variables inside strings.

1. Using f-strings

The simplest and most preferred way to format strings is by using f-strings.

Example: Embed variables directly using {} placeholders.

```
name = "Alice"
age = 22
print(f"Name: {name}, Age: {age}")
```

Output

```
Name: Alice, Age: 22
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

2. Using format()

Another way to format strings is by using format() method.

Example: Use placeholders {} and pass values positionally.

```
s = "My name is {} and I am {} years old.".format("Alice", 22)
print(s)
```

Output

```
My name is Alice and I am 22 years old.
```

String Membership Testing

in keyword checks if a particular substring is present in a string.

Example: Here, we are testing for the presence of substrings.

```
s = "GeeksforGeeks"
print("Geeks" in s)
print("GfG" in s)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

List Data Structure:

-->If we want to represent a group of individual objects as a single entity where insertion order preserved and duplicates are allowed, then we should go for list.

-->Insertion order is preserved.

-->Duplicates objects are allowed.

-->Heterogeneous objects are allowed.

-->List is dynamic because based on our requirement we can increase the size and

decrease the size.

-->In the list elements will be palced with in square brackets and with comma separator.

-->We can differentiate duplicate elements by using index and we can preserve insertion

order by using index.Hence index will play very important role.

-->Python supports both +ve and -ve indexes. +ve index means from left to right where as -ve index means right to left

D Y PATIL UNIVERSITY
NAVI MUMBAI

Ex:

```
l=[10,"A","B",20,30,True]

-6  -5 -4 -3  -2   -1
10  A  B  20  30  True
0   1  2  3   4   5
```

-->List objects are mutable. i.e we can change the content.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Creation of list objects:

==========

1).We can create empty list objects as follows.....

```
1)  list=[ ]
2)  print(list)
3)  print(type(list))
```

2).If we know elements already then we can create list as follows.

```
1)  list=[10,20,30,40]
```

3).With dynamic input:

```
1)  list=eval(input("Enter List:"))
2)  print(list)
3)  print(type(list))
```


IL
Y
NAVI MUMBAI

4).with list() function:

```
1)  l=list(range(10))
2)  print(l)
3)  print(type(l))
```

Ex:

```
1)  s="mahesh"
2)  l=list(s)
3)  print(l)
```

5).with split() function:

```
1)  s="Learning Python Is Very Easy"
2)  l=s.split()
3)  print(type(l))
4)  print(l)
```

Note:

Sometimes we can create list inside another list, such type of lists are called as nested lists.

Ex: **[10,20,[30,40]]**

D Y PATIL UNIVERSITY
NAVI MUMBAI

Accessing elements of List:

-->We can access elements of the list either by using index or by using slice operator(:)

1).By using Index:

-->List follows zero based index. i.e index of first element is zero.

-->List supports both +ve & -ve indexes. +ve(left to right). -ve(Right to left)

```
1)  list=[10,20,30,40]
2)
3)  print(list[0])==>10
4)  print(list[-1])==>40
5)  print(list[10])==>List index out of range
```

2).By using slice operator:

Syn:

list2=list1[start:stop:step]

start-->It indicates the index where slice has to start.

default value is 0.

stop-->It indicates the index where slice has to end.

default value is max alloed index of lisr i.e length of the list.

step-->increment value

default value is 1.

D Y PATIL UNIVERSITY NAVI MUMBAI

Example

Ex:

```
1)  n=[1,2,3,4,5,6,7,8,9,10]
2)  print(n[2:7:2])
3)  print(n[4::2])
4)  print(n[3:7])
5)  print(n[8:2:-2])
6)  print(n[4:100])
```

```
o/p:D:\pythonclasses>py test.py
        [3, 5, 7]
        [5, 7, 9]
        [4, 5, 6, 7]
        [9, 7, 5]
        [5, 6, 7, 8, 9, 10]
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

List Vs mutability:

==========

-->Once we create a list object, we can modify its content. Hence list objects are mutable.

Ex:

```
1)  n=[10,20,30,40]
2)  print(n)
3)  n[1]=333
4)  print(n)
```

```
o/p:[10,20,30,40]
    [10,333,30,40]
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Traversing the elements of list:

-->The sequential access of each element in the list is called as traversal.

1).By using while loop:

```
n=[0,1,2,3,4,5,6,7,8,9,10]
i=0
while i<len(n):
  print(n[i])
  i=i+1
```

2).By using for loop:

```
n=[0,1,2,3,4,5,6,7,8,9,10]
for i in n:
  print(i)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

3).To display only even numbers:

```
1)  n=[0,1,2,3,4,5,6,7,8,9,10]
2)  for i in n:
3)   if i%2==0:
4)            print(i)
```

4).To display elements by index wise:

```
1)  l=["A","B","C"]
2)  x=len(l)
3)  for i in range(x):
4)   print(l[i], "is available at +ve index: ",i,"and at -ve index: ",i-x)
```

```
o/p:D:\pythonclasses>py test.py
A is available at +ve index:  0 and at -ve index:  -3
B is available at +ve index:  1 and at -ve index:  -2
C is available at +ve index:  2 and at -ve index:  -1
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Important functions of List:

==========

1).To get the information about list:

1.len():

Returns number of elements present in the list.

Ex:

```
1)  l=[10,20,30,40]
2)  len(l)==>4
```

2.count():

It returns the number of occurences of specified item in the list

Ex:

```
1)  n=[1,2,2,2,2,3,3]
2)  print(n.count(1))==>1
3)  print(n.count(2))==>4
4)  print(n.count(3))==>2
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

3.index():

It returns index of first occurance of the specified element.

Ex:

```
1)  l=[1,2,2,3,3,4]
2)  print(l.index(1))==>0
3)  print(l.index(2))==>1
4)  print(l.index(3))==>3
5)  print(l.index(4))==>5
```

Note:

If the specified element not present in the list then we will get ValueError. Hence before index() method we have to check whether item present in the list or not by using in operator.

D Y PATIL UNIVERSITY
NAVI MUMBAI

2).Manipulating elements of list:

==========

1). append() function:

We can use append() function to add item at the end of the list.

Ex:

```
1)  >>> l=[ ]
2)  >>> l    [ ]
3)  >>> l.append("A")
4)  >>> l.append("B")
5)  >>> l.append("C")
6)  >>> l    ['A', 'B', 'C']
```

Ex: To add all elements to list upto 100 which are divisible by 10

```
1)  list=[ ]
2)  for i in range(101):
3)   if i%10==0:
4)           list.append(i)
5)  print(list)
```

o/p:D:\pythonclasses>py test.py
[0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

D Y PATIL UNIVERSITY
NAVI MUMBAI

2).insert() function:

To insert an item at specified index position.

Ex:

```
1)  >>> n=[1,2,3,4]
2)  >>> n.insert(1,333)
3)  >>> n   [1, 333, 2, 3, 4]
```

Ex:

```
1)  >>> n=[1,2,3,4]
2)  >>> n.insert(10,777)
3)  >>> n.insert(-10,999)
4)  >>>print(n)     [999, 1, 2, 3, 4, 777]
```

Note:

If the specified index is greater than max index then element will be inserted at last position. If the specified index is smaller than min index then element will be inserted at first position.

D Y PATIL UNIVERSITY
NAVI MUMBAI

3).extend() function:

-->To add all items of one list to another list.

Syn: l1.extend(l2)

-->All items present in l2 will be added to l1.

Ex:

```
1)  order1=["Chicken","Mutton","Fish"]
2)  order2=["RC","KF","FO"]
3)  order1.extend(order2)
4)  print(order1)
```

```
o/p:D:\pythonclasses>py test.py
['Chicken', 'Mutton', 'Fish', 'RC', 'KF', 'FO']
```

Ex:

```
1)  order=["Chicken","Mutton","Fish"]
2)  order.extend("mushroom")
3)  print(order)
```

```
o/p:D:\pythonclasses>py test.py
['Chicken', 'Mutton', 'Fish', 'm', 'u', 's', 'h', 'r', 'o', 'o', 'm']
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

4).remove() function:

We can use this function to remove specified item from the list. If the item present multiple times then only first occurence will be removed.

Ex:

```
1) n=[10,20,30,10,30]
2) n.remove(10)
3) print(n)
```

o/p:D:\pythonclasses>py test.py
[20, 30, 10, 30]

-->If the specified item not present in the list we will get ValueError

Ex:

```
1) n=[10,20,30]
2) n.remove(50)
3) print(n)
```

o/p:ValueError:list.remove(x):x not in list

D Y PATIL UNIVERSITY
NAVI MUMBAI

5).pop() function:

-->It removes and returne the last element of the list.

-->This is only the function which manipulates list and returns some element.

Ex:

```
1) n=[10,20,30,40]
2) print(n.pop())
3) print(n.pop())
4) print(n)
```

```
o/p:D:\pythonclasses>py test.py
40
30
[10, 20]
```

-->If the list is empty then pop() function raises indexerror.

Ex:

```
1) n=[ ]
2) print(n.pop())
```

```
o/p:IndexError: pop from empty list
```

Note:

1.pop() is the only function which manipulates the list and returns some value.

2.In general we can use append() and pop() functions to implement stack datastructure by using list, which follows LIFO(Last In First Out) order.

D Y PATIL UNIVERSITY NAVI MUMBAI

Ordering elements of the List:

1.reverse():

We can use reverse() order of elements of list.

Ex:

```
1)      n=[10,20,30,40]
2)      n.revrse()
3)      print(n)
```

o/p: [40,30,20,10]

2.sort() function:

In list by default insertion order is preserved. If we want to sort the elements of list according to default natural sorting order then we should go for sort() method.

For Numbers==>default sording order is ascending order.
For Strings==>default sorting order is alphabatical order.

Ex:

```
1)      n=[20,10,40,30]
2)      n.sort()
3)      print(n)
```

o/p:[10,20,30,40]

D Y PATIL UNIVERSITY
NAVI MUMBAI

Tuples

- A tuple in Python is an immutable ordered collection of elements.
- Tuples are similar to lists, but unlike lists, they cannot be changed after their creation (i.e., they are immutable).
- Tuples can hold elements of different data types.
- The main characteristics of tuples are being ordered, heterogeneous and immutable.

Creating a Tuple:

- A tuple is created by placing all the items inside parentheses (), separated by commas.
- A tuple can have any number of items and they can be of different data types.

```
tup = ()
print(tup)

# Using String
tup = ('Geeks', 'For')
print(tup)

# Using List
li = [1, 2, 4, 5, 6]
print(tuple(li))

# Using Built-in Function
tup = tuple('Geeks')
print(tup)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Creating a Tuple with Mixed Datatypes.

Tuples can contain elements of various data types, including other tuples, lists, dictionaries and even functions.

```
tup = (5, 'Welcome', 7, 'Geeks')
print(tup)

# Creating a Tuple with nested tuples
tup1 = (0, 1, 2, 3)
tup2 = ('python', 'geek')
tup3 = (tup1, tup2)
print(tup3)

# Creating a Tuple with repetition
tup1 = ('Geeks',) * 3
print(tup1)

# Creating a Tuple with the use of loop
tup = ('Geeks')
n = 5
for i in range(int(n)):
    tup = (tup,)
    print(tup)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Python Tuple Basic Operations

Below are the Python tuple operations.

- Accessing of Python Tuples
- Concatenation of Tuples
- Slicing of Tuple
- Deleting a Tuple

Accessing of Tuples

- We can access the elements of a tuple by using indexing and slicing, similar to how we access elements in a list.
- Indexing starts at 0 for the first element and goes up to n-1, where n is the number of elements in the tuple.
- Negative indexing starts from -1 for the last element and goes backward.

```
# Accessing Tuple with Indexing
tup = tuple("Geeks")
print(tup[0])

# Accessing a range of elements using slicing
print(tup[1:4])
print(tup[:3])

# Tuple unpacking
tup = ("Geeks", "For", "Geeks")

# This line unpack values of Tuple1
a, b, c = tup
print(a)
print(b)
print(c)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Concatenation of Tuples

Tuples can be concatenated using the + operator. This operation combines two or more tuples to create a new tuple.

Note: Only the same datatypes can be combined with concatenation, an error arises if a list and a tuple are combined.

```
tup1 = (0, 1, 2, 3)
tup2 = (‘John', ‘Jiya', ‘Jimmy)

tup3 = tup1 + tup2
print(tup3)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Slicing of Tuple

Slicing a tuple means creating a new tuple from a subset of elements of the original tuple. The slicing syntax is tuple[start:stop:step].

Note- Negative Increment values can also be used to reverse the sequence of Tuples.

```
tup = tuple(‘WelcomeToDYPatil')

# Excluding First element
print(tup[1:])

# Reversing the Tuple
print(tup[::-1])

# Printing elements of a Range
print(tup[4:9])
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Deleting a Tuple

Since tuples are immutable, we cannot delete individual elements of a tuple. However, we can delete an entire tuple using del statement.

Note: *Printing of Tuple after deletion results in an Error.*

```
tup = (0, 1, 2, 3, 4)
del tup

print(tup)
```

Output

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 6, in <module>
NameError: name 'tup' is not defined
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Tuple Unpacking with Asterisk (*)

In Python, the " * " operator can be used in tuple unpacking to grab multiple items into a list. This is useful when you want to extract just a few specific elements and collect the rest together.

```
tup = (1, 2, 3, 4, 5)

a, *b, c = tup

print(a)
print(b)
print(c)
```

Output

```
1
[2, 3, 4]
5
```

Explanation:

- **a** gets the first item.
- **c** gets the last item.
- ***b** collects everything in between into a list.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Sets in Python

A Set in Python is used to store a collection of items with the following properties.

- No duplicate elements. If try to insert the same item again, it overwrites previous one.
- An unordered collection. When we access all items, they are accessed without any specific order and we cannot access items using indexes as we do in lists.
- Internally use hashing that makes set efficient for search, insert and delete operations. It gives a major advantage over a list for problems with these operations.
- Mutable, meaning we can add or remove elements after their creation, the individual elements within the set cannot be changed directly.

Example: This example shows how to create a set with some numbers and check its type.

```
s = {10, 50, 20}
print(s)
print(type(s))
```

Output

```
{10, 50, 20}
<class 'set'>
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Type Casting

set() method in python is used to convert other data types, such as lists or tuples, into sets.

```
# typecasting list to set
s = set(["a", "b", "c"])
print(s)

# Adding element to the set
s.add("d")
print(s)
```

Output

```
{'c', 'b', 'a'}
{'d', 'c', 'b', 'a'}
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Check unique and Immutable

Python sets cannot have duplicate values. While you cannot modify the individual elements directly, you can still add or remove elements from the set.

```
# a set cannot have duplicate values
s = {“John", "for", “John"}
print(s)

# values of a set cannot be changed
s[1] = "Hello"
print(s)
```

Output

{‘John', 'for'}

TypeError: 'set' object does not support item assignment

Explanation:

s = {“John", "for", “John"}duplicates are removed, only unique values remain.

s[1] = "Hello" Error, set elements cannot be changed directly. Only add() or remove() can modify a set.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Heterogeneous Element

Python sets can store heterogeneous elements in it, i.e., a set can store a mixture of string, integer, boolean, etc datatypes.

```
s = {"Geeks", "for", 10, 52.7, True}
print(s)
```

Output

```
{True, 52.7, 'Geeks', 10, 'for'}
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Frozen Sets

A frozenset is an immutable version of a set. Its elements cannot be changed after creation, but you can perform operations like union, intersection and difference. Use frozenset() to create one.

```
# Normal set (mutable)
s = set(["a", "b", "c"])
print("Normal Set:", s)

# Frozen set (immutable)
fs = frozenset(["e", "f", "g"])
print("Frozen Set:", fs)
```

Output

```
('Normal Set:', set(['a', 'c', 'b']))
('Frozen Set:', frozenset(['e', 'g', 'f']))
```

***Note**: Frozensets are immutable, so methods like add() or remove() cannot be used. They are also hashable, which allows them to be used as dictionary keys.*

D Y PATIL UNIVERSITY
NAVI MUMBAI

Methods for Sets

1. Adding elements to Sets: add() function is used to insert new elements into a set. It automatically ignores duplicates.

```
s = {"a", "b", "c"}
s.add("d")
print(s)
```

Output

```
{'c', 'd', 'a', 'b'}
```

2. Union of Sets: union() function combines two sets and returns a new set with all unique elements.

```
a = {"x", "y"}
b = {"y", "z"}
u = a.union(b)
print(u)
```

Output

```
{'z', 'y', 'x'}
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

3. Intersection of Sets: intersection() function returns a new set containing elements that are common to both sets.

```
a = {1, 2, 3}
b = {2, 3, 4}
i = a.intersection(b)
print(i)
```

Output

```
{2, 3}
```

4. Difference of Sets: difference() function returns a set containing elements that are in the first set but not in the second.

```
a = {1, 2, 3}
b = {2, 3, 4}
d = a.difference(b)
print(d)
```

Output

```
{1}
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

5. Clearing a Set: clear() function removes all elements from a set, leaving it empty.

```
s = {1, 2, 3}
s.clear()
print(s)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Python Dictionary

A Python dictionary is a data structure that stores data in key-value pairs, where each key is unique and is used to retrieve its associated value. It is mainly used when you want to store and access data by a name (key) instead of by position like in a list.

Example: This example shows how a dictionary stores data using keys and values.

```
data = { "name": "Jake", "age": 22 }
print(data)
```

Output

```
{'name': 'Jake', 'age': 22}
```

- "name" and "age" are keys
- "Jake" and 22 are their values
- dictionary stores data in key : value format

D Y PATIL UNIVERSITY
NAVI MUMBAI

Creating a Dictionary

A dictionary is created by writing key-value pairs inside { }, where each key is connected to a value using colon (:). A dictionary can also be created using the dict() function.

```
d1 = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print(d1)

# using dict() constructor
d2 = dict(a = "Geeks", b = "for", c = "Geeks")
print(d2)
```

Output

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
{'a': 'Geeks', 'b': 'for', 'c': 'Geeks'}
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Accessing Dictionary Items

A value in a dictionary is accessed by using its key. This can be done either with square brackets [ ] or with the get() method. Both return the value linked to the given key.

```
d = { "name": "Kat", 1: "Python", (1, 2): [1,2,4] }

# Access using key
print(d["name"])

# Access using get()
print(d.get("name"))
```

Output

```
Kat
Kat
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Adding and Updating Dictionary Items

New items are added to a dictionary using the assignment operator (=) by giving a new key a value. If an existing key is used with the assignment operator, its value is updated with the new one.

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks'}

# Adding a new key-value pair
d["age"] = 22

# Updating an existing value
d[1] = "Python dict"
print(d)
```

Output

```
{1: 'Python dict', 2: 'For', 3: 'Geeks', 'age': 22}
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Removing Dictionary Items

Dictionary items can be removed using built-in deletion methods that work on keys:

- **del**: removes an item using its key
- **pop()**: removes the item with the given key and returns its value
- **clear()**: removes all items from the dictionary
- **popitem()**: removes and returns the last inserted key–value pair

```
d = {1: 'Jim', 2: 'For', 3: 'John', 'age':22}
# Using del
del d["age"]
print(d)
# Using pop()
val = d.pop(1)
print(val)
# Using popitem()
key, val = d.popitem()
print(f"Key: {key}, Value: {val}")
# Using clear()
d.clear()
print(d)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Iterating Through a Dictionary

A dictionary can be traversed using a for loop to access its keys, values or both key-value pairs by using the built-in methods keys(), values() and items().

```
d = {1: 'Geeks', 2: 'For', 'age':22}

# Iterate over keys
for key in d:
    print(key)

# Iterate over values
for value in d.values():
    print(value)

# Iterate over key-value pairs
for key, value in d.items():
    print(f"{key}: {value}")
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Thank You

D Y PATIL
UNIVERSITY
NAVI MUMBAI